



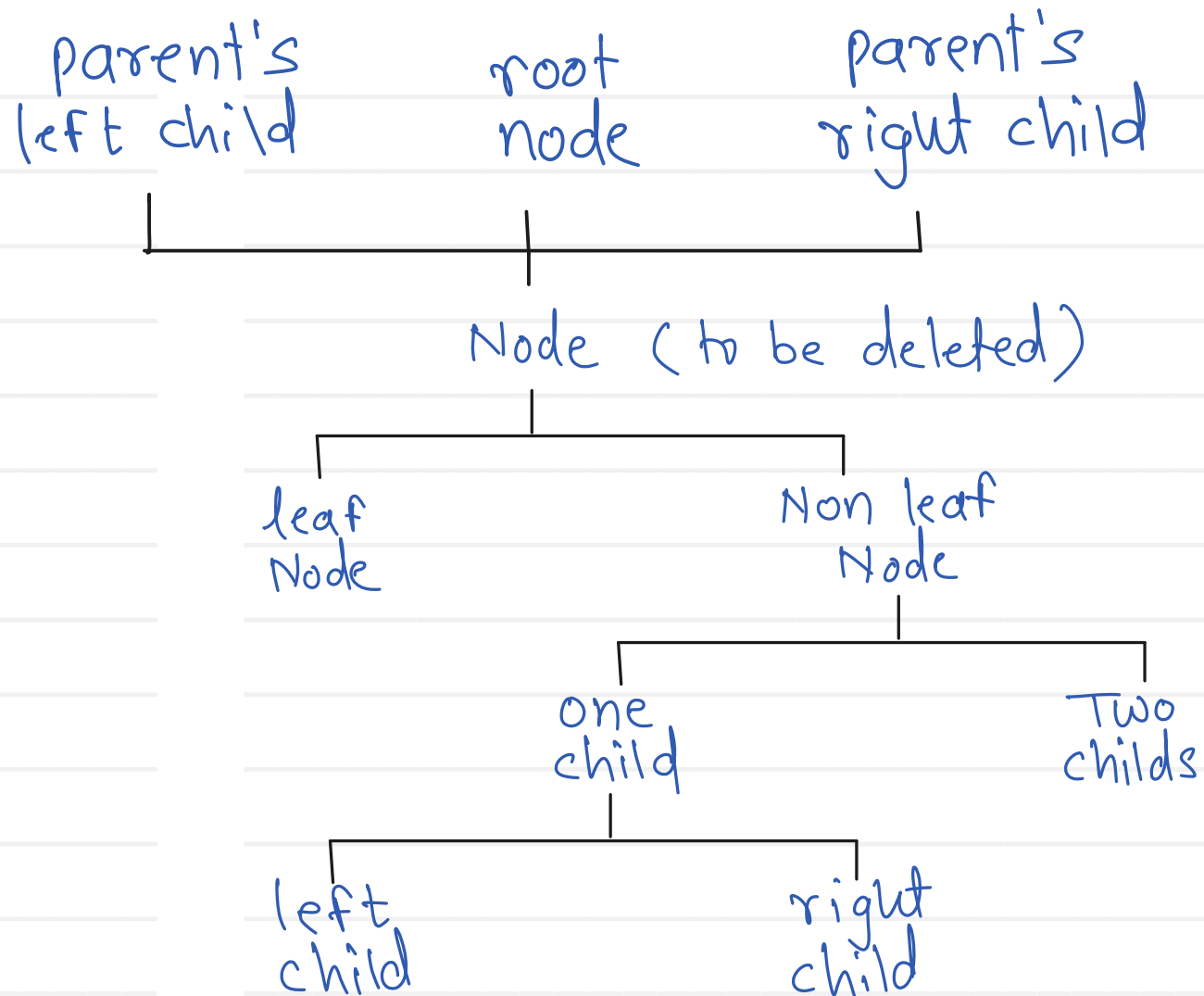
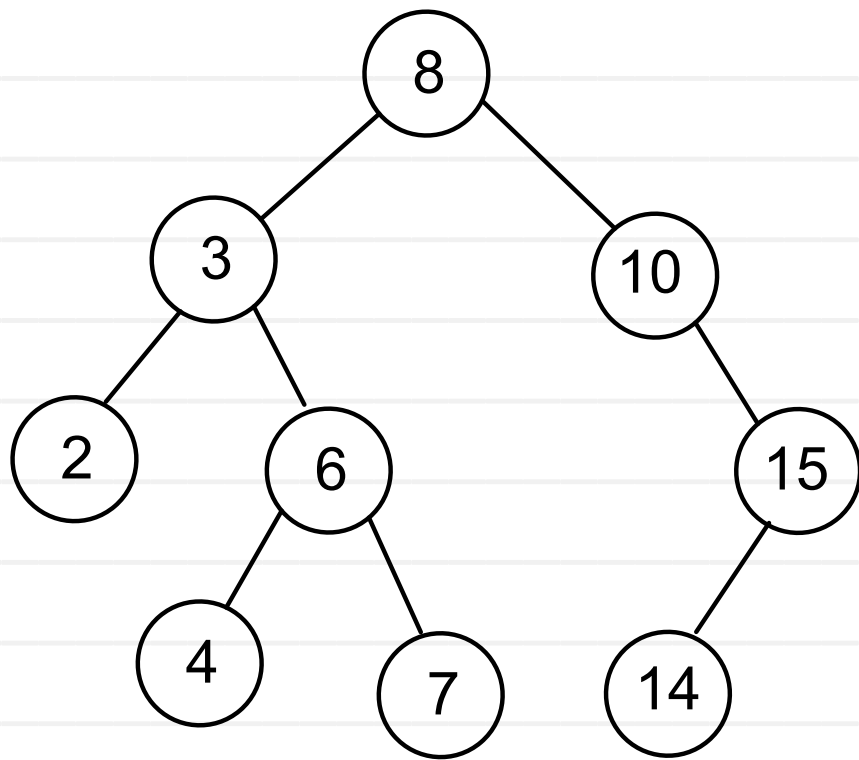
Sunbeam Institute of Information Technology
Pune and Karad

Algorithms and Data structures

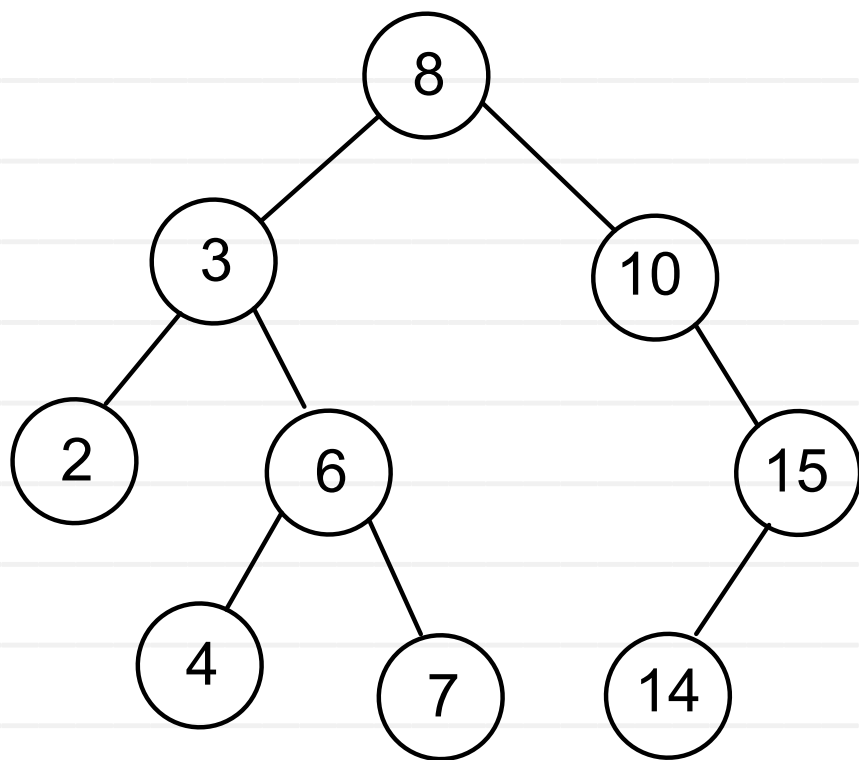
Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

Binary Search Tree - Delete Node



Binary Search Tree - Binary Search with Parent



key = 7

trav	parent
\$8	null
\$3	\$8
\$6	\$3
\$7	\$6

key = 8

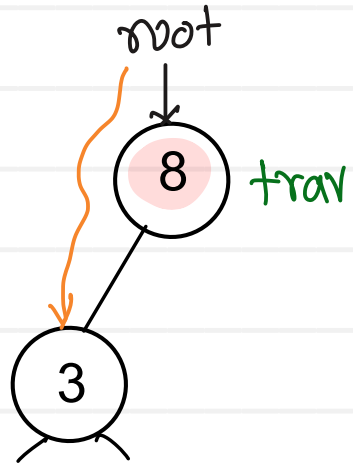
trav	parent
\$8	null

key = 18

trav	parent
\$8	null
\$10	\$8
\$15	\$10
null	\$15

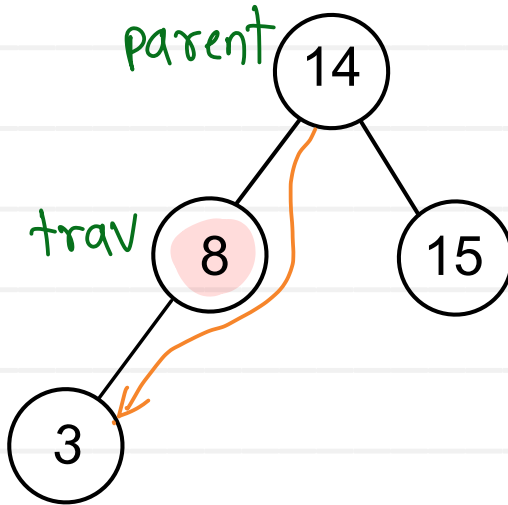
BST- Delete Node with Single child node (Left child)

① root Node



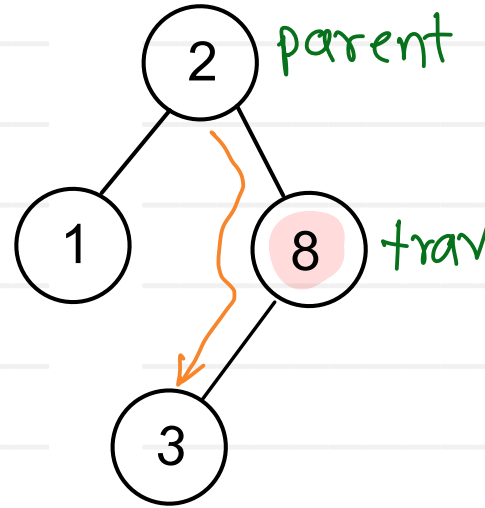
$root = trav.left$

② parent's left child



$parent.left = trav.left$

③ parent's right child

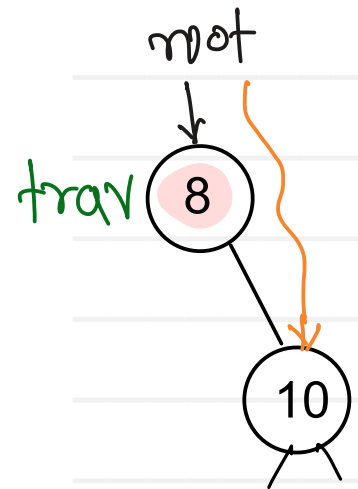


$parent.right = trav.left$

1. check if has only left child
2. if it is root node
then update root by its left
3. if it is parent's left child
then update parent's left by left child
4. if it is parent's right child
then update parent's right by left child

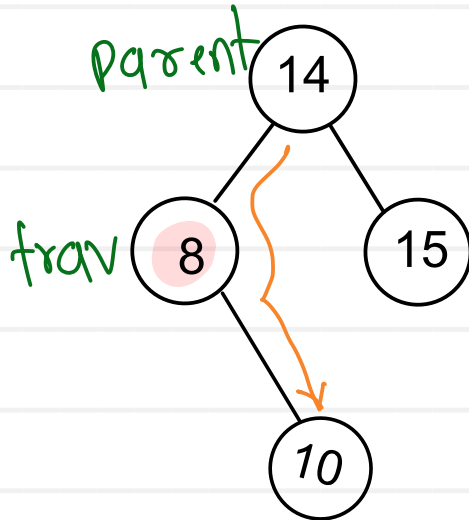
BST - Delete Node with Single child node (Right child)

(a) root node



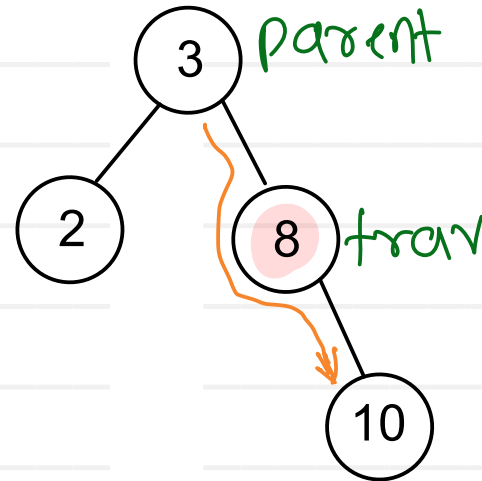
$root = trav.right$

(b) Parent's left child



$parent.left = trav.right$

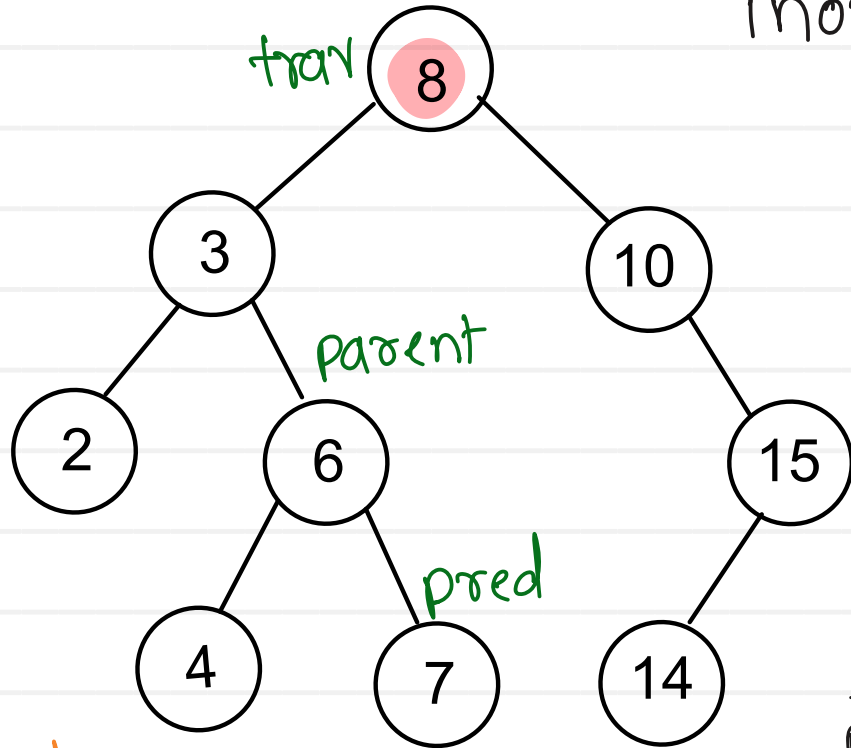
(c) Parent's right child



$parent.right = trav.right$

1. check if has only right child
2. if it is root node
then update root by its right child
3. if it is parent's left child
then update parent's left by right child
4. if it is parent's right child
then update parent's right by right child

BST - Delete Node with Two child node



inorder : 2 3 4 6 7 8 10 14 15

inorder
predecessor

1. go into left subtree
2. find extreme right

inorder
successor

1. go into right subtree
2. find extreme left

1. find inorder predecessor
2. replace value of node to be deleted by its predecessor value
3. delete predecessor from BST

```

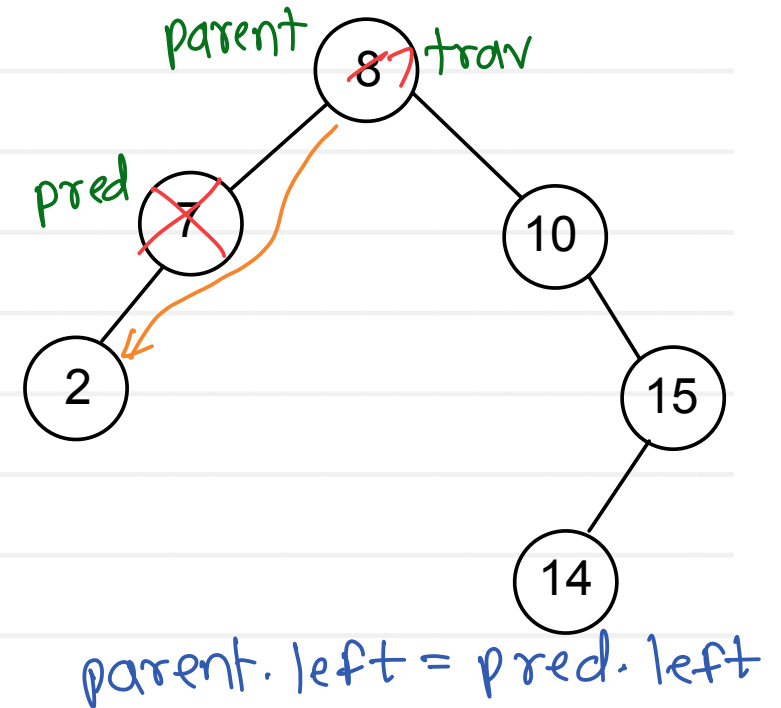
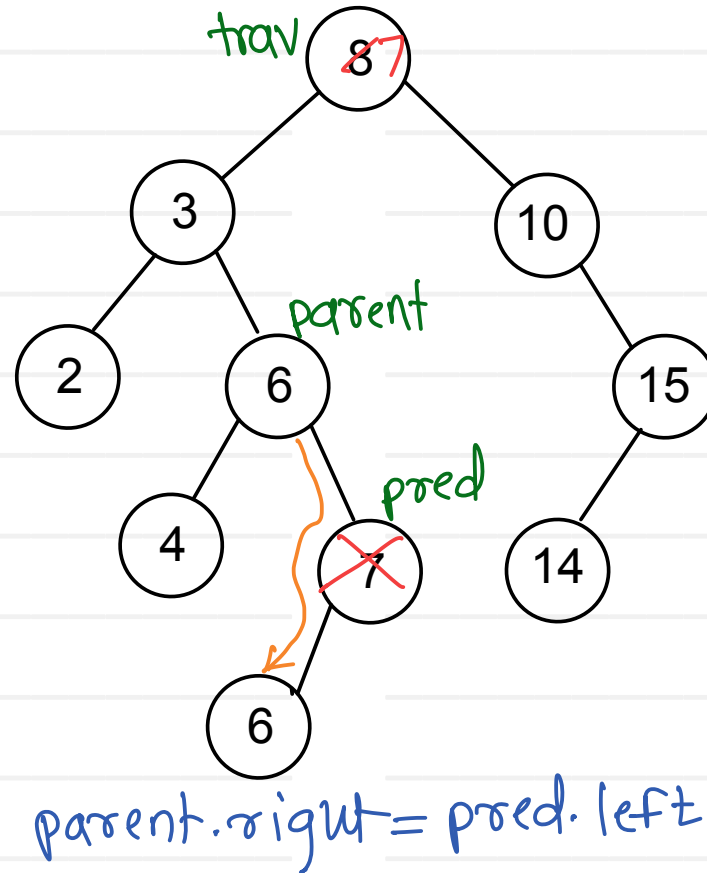
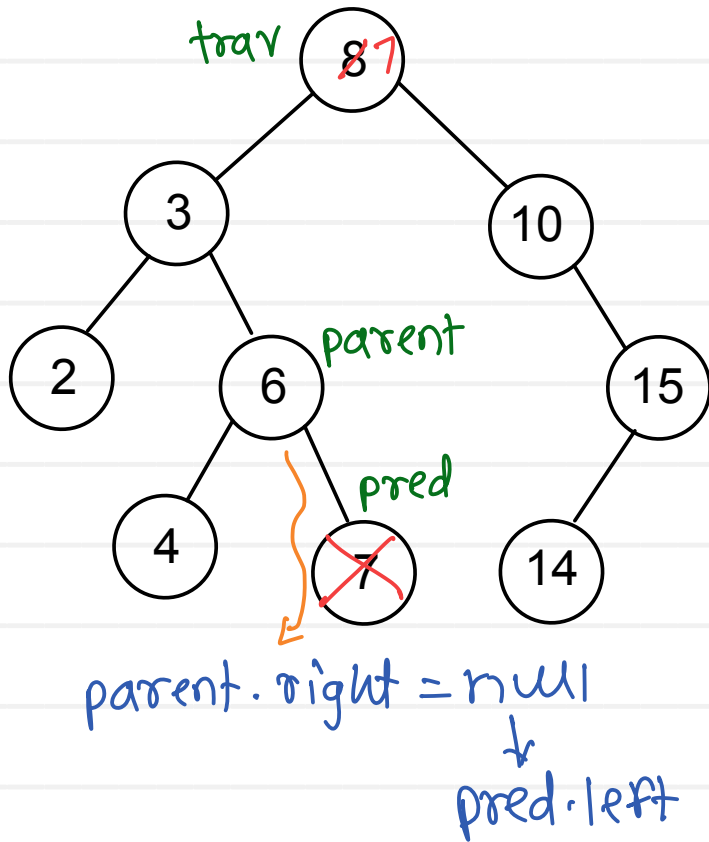
pred = trav.left
parent = trav;
while (pred.right != null) {
    parent = pred;
    pred = pred.right;
}

```

pred parent

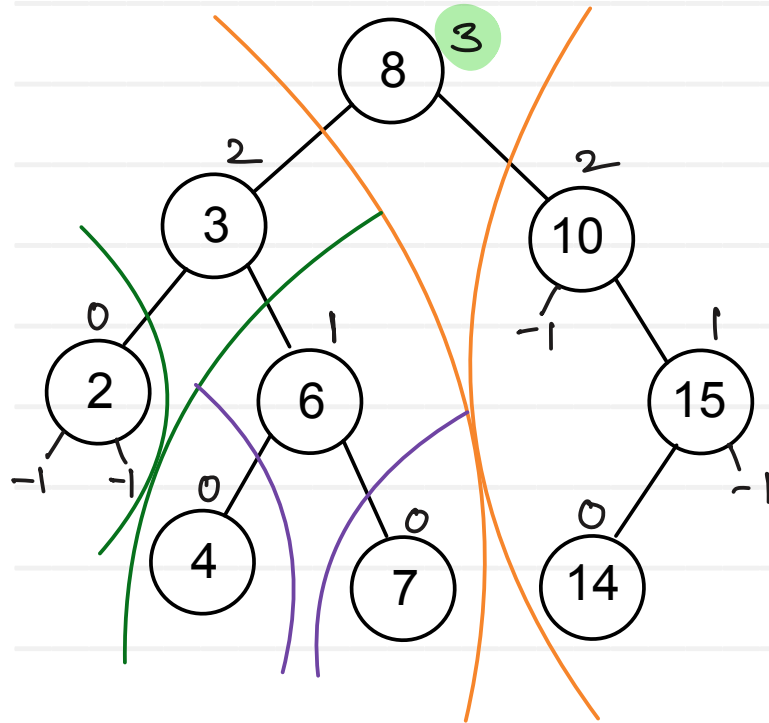
\$3	\$8
\$6	\$3
\$7	\$6

Delete with two childs



Binary Search Tree - Height

Height of root = MAX (height (left sub tree), height (right sub tree)) + 1

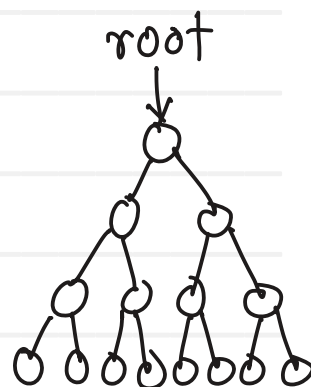


1. If left or right sub tree is absent then return -1
2. Find height of left sub tree
3. Find height of right sub tree
4. Find max height
5. Add one to max height and return

BST - Time complexity of operations

Capacity : max number of nodes for given height.

height (h)	Capacity (n)
-1	0
0	1
1	3
2	7
3	15



$$n = 2^{h+1} - 1$$

$$n = 2^{h+1} - 1$$

$$2^{h+1} = n + 1$$

$$2^h \approx n$$

$$\log 2^h = \log n$$

$$h = \frac{\log n}{\log 2}$$

Time $\propto$ height
Time $\propto \frac{\log n}{\log 2}$

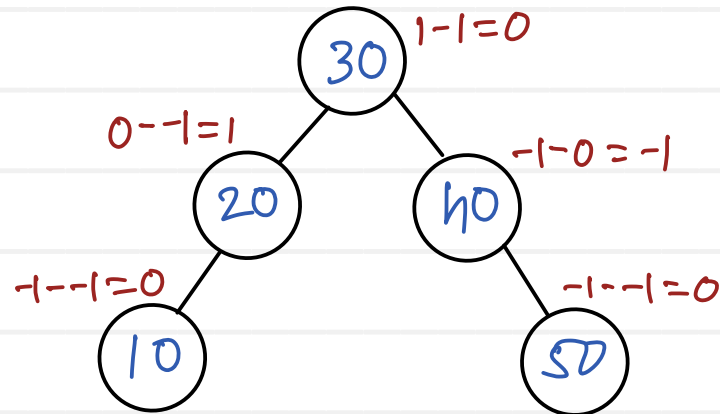
$$T(n) = O(\log n)$$

Add
search
Delete

$$T(n) = O(n) - \text{traverse}$$

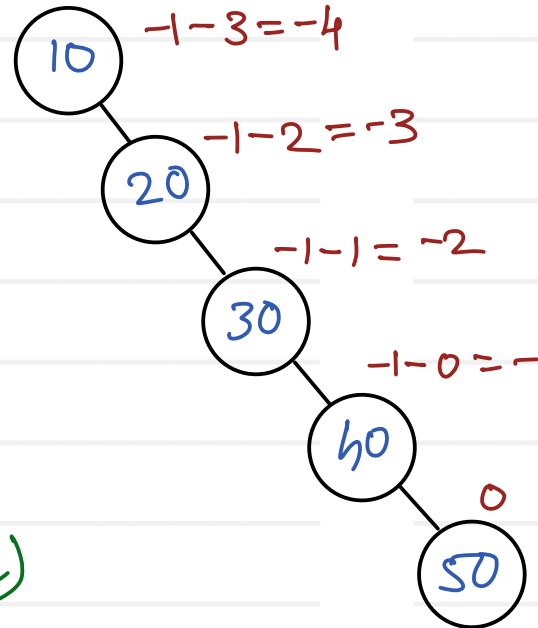
Skewed Binary Search Tree

Keys : 30, 40, 20, 50, 10



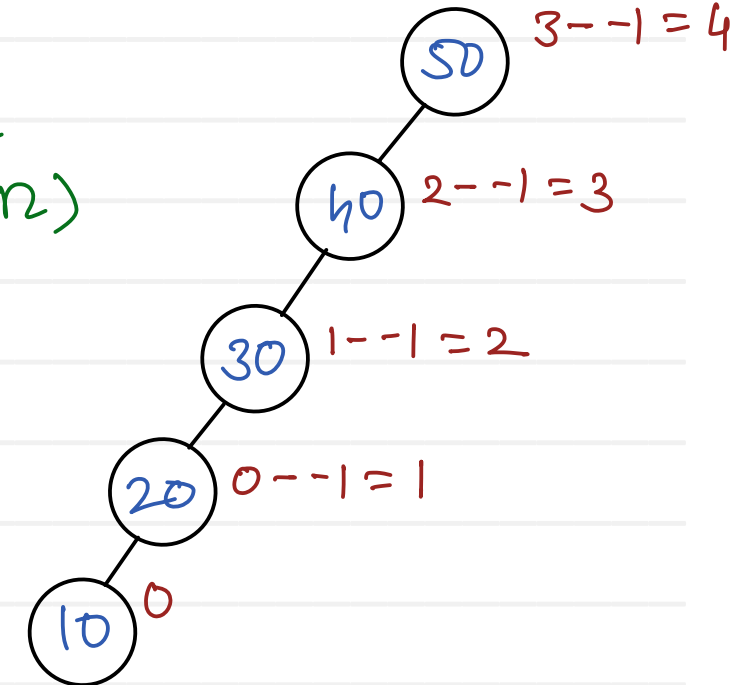
height $\approx \log n$
 $T(n) = O(\log n)$

Keys : 10, 20, 30, 40, 50



height $\approx n$
 $T(n) = O(n)$

Keys : 50, 40, 30, 20, 10



- In binary tree if only left or right links are used, tree grows only in one direction such tree is called as skewed binary search tree
 - Left skewed binary search tree
 - Right skewed binary search tree
- Time complexity of any BST is $O(h)$
- Skewed BST have maximum height ie same as number of elements.
- Time complexity of searching in skewed BST is $O(n)$

Balanced BST

- To speed up searching, height of BST should be minimum as possible
- If nodes in BST are arranged, so that its height is kept as less as possible, is called as Balanced BST

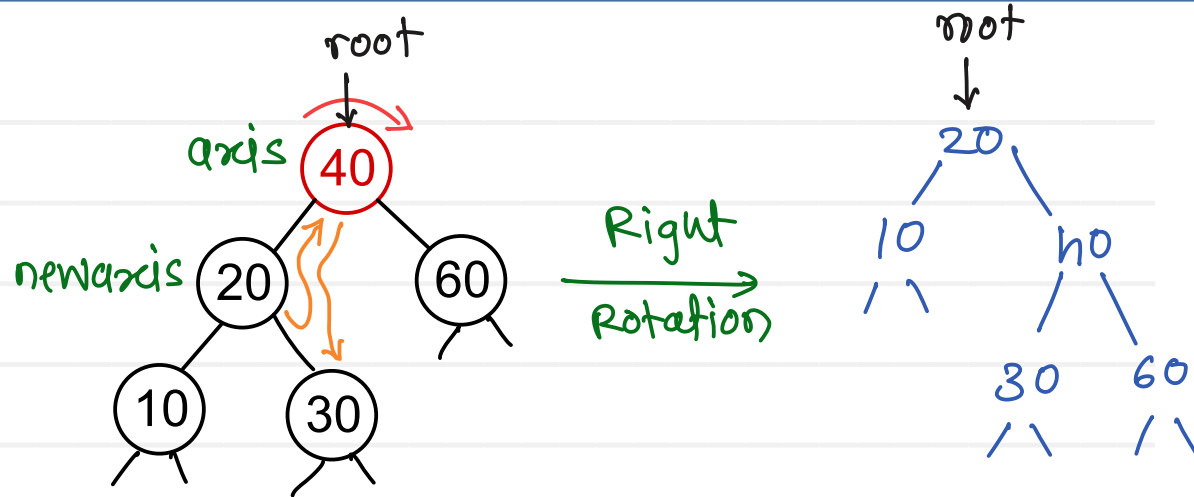
$$\text{Balance factor} = \text{Height (left sub tree)} - \text{Height (right sub tree)}$$

- tree is balanced if balance factors of all the nodes is either -1, 0 or +1
- balance factors = $\{-1, 0, +1\}$
- A tree can be balanced by applying series of left or right rotations on imbalance nodes (node having balance factor other than -1, 0 or +1)

- types of balanced BST :

1. AVL tree
2. Red Black tree
3. Splay tree

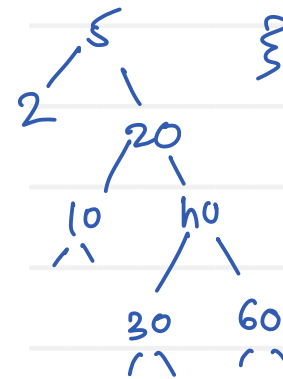
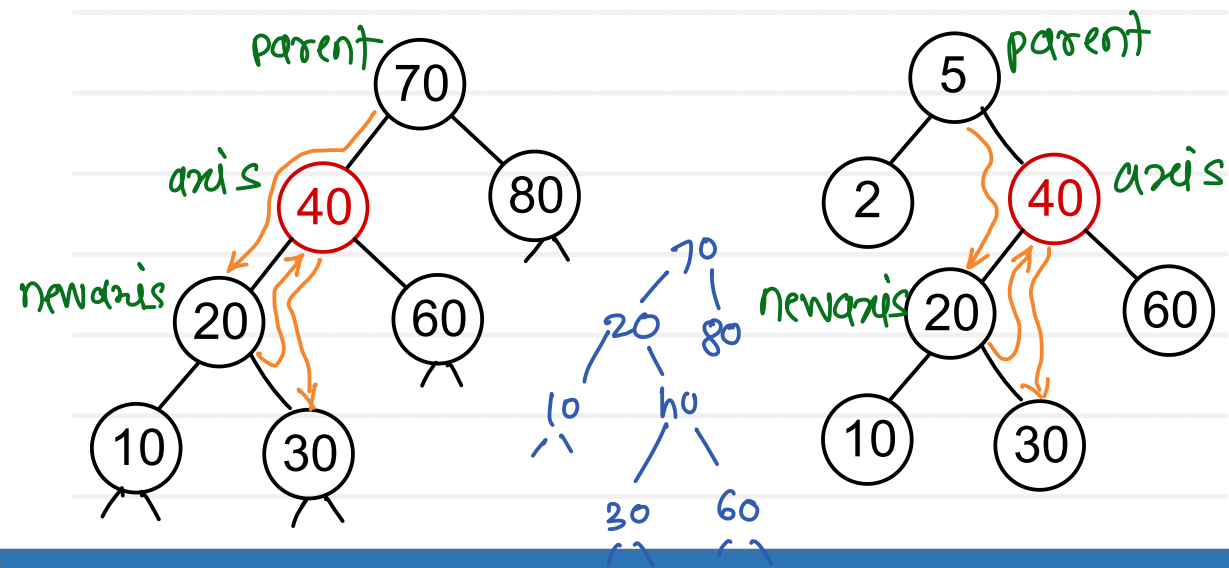
Right Rotation



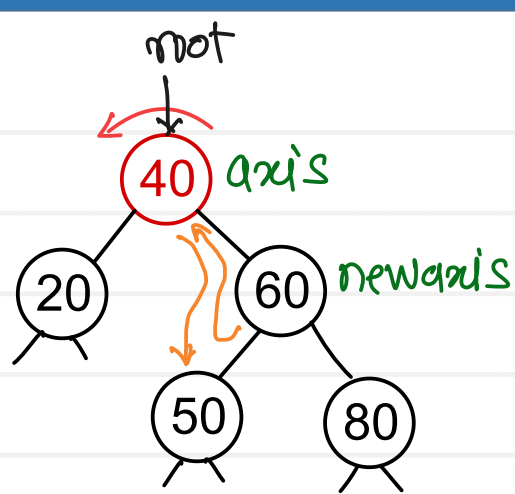
```

rightRotation(axis, parent) {
    newaxis = axis.left;
    axis.left = newaxis.right;
    newaxis.right = axis;
    if (axis == root)
        root = newaxis;
    else if (axis == parent.left)
        parent.left = newaxis;
    else if (axis == parent.right)
        parent.right = newaxis;
}

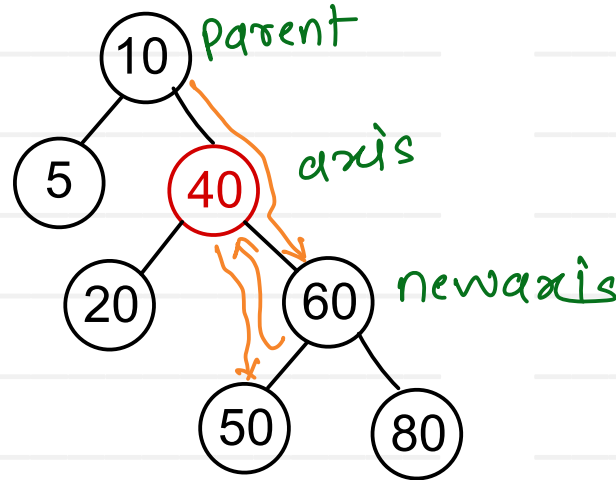
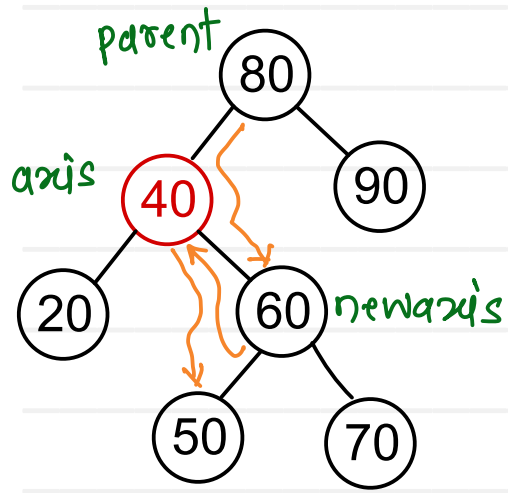
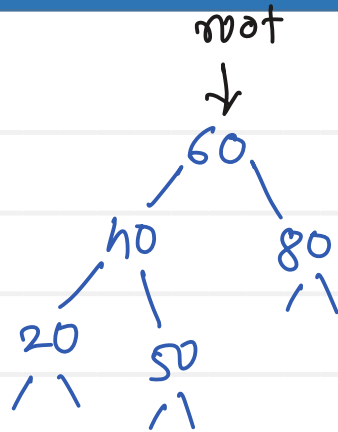
```



Left Rotation



Left
Rotation



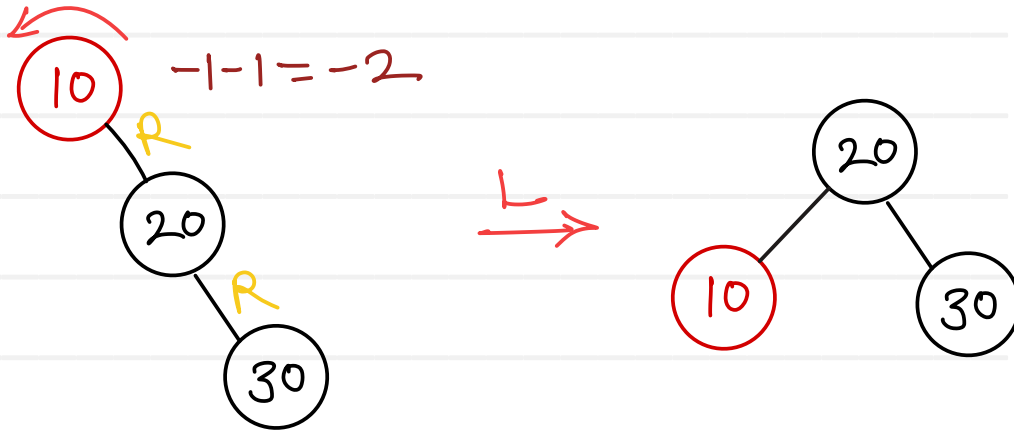
```

leftRotation(axis, parent) {
    newaxis = axis.right
    axis.right = newaxis.left;
    newaxis.left = axis;
    if (axis == root)
        root = newaxis;
    else if (axis == parent.left)
        parent.left = newaxis;
    else if (axis == parent.right)
        parent.right = newaxis;
}
    
```

Rotation cases (Single Rotations)

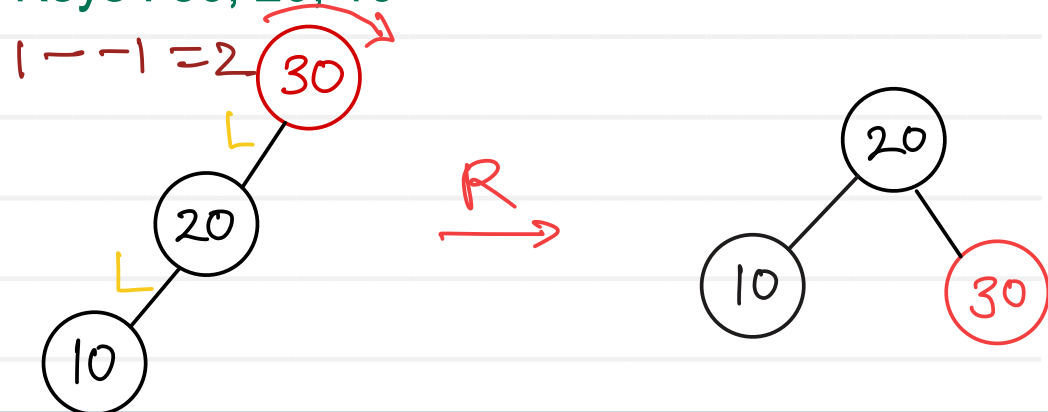
RR Imbalance : apply left rotation on imbalance node

Keys : 10, 20, 30



LL Imbalance : apply right rotation on imbalance node

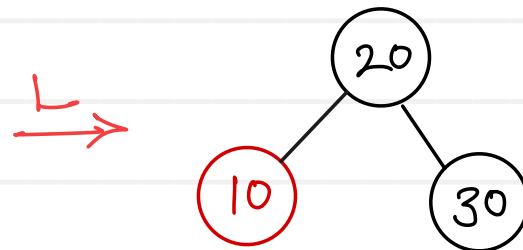
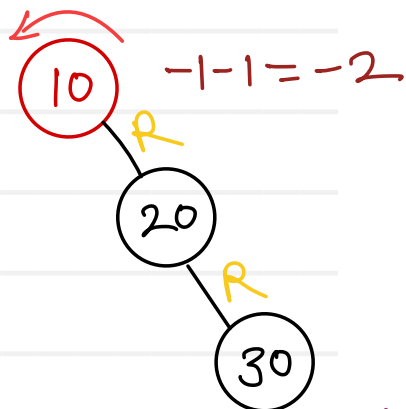
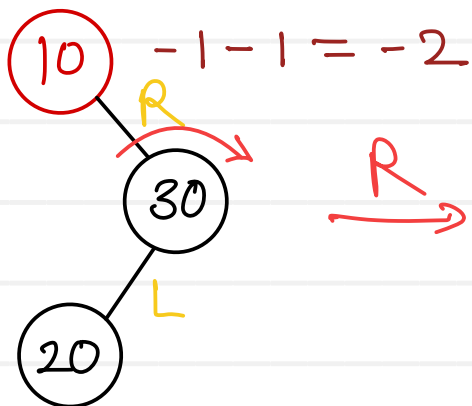
Keys : 30, 20, 10



Rotation cases (Double Rotations)

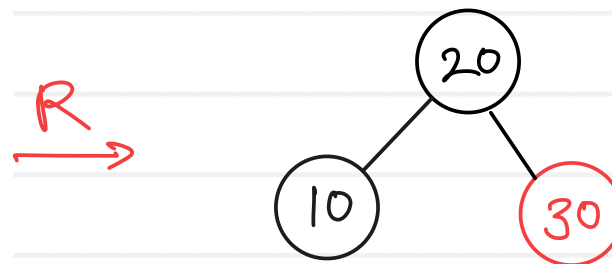
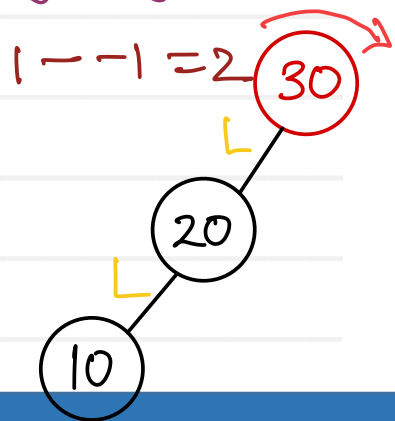
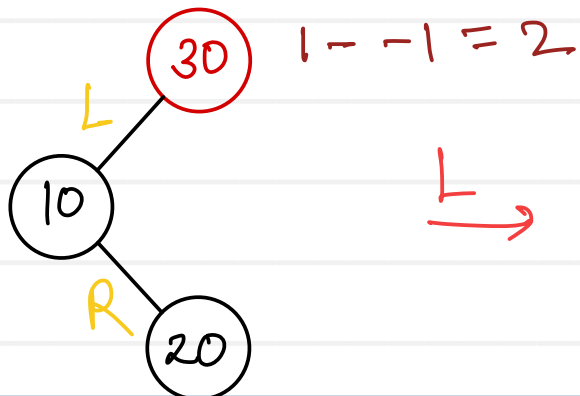
RL Imbalance : apply right rotation on right of imbalance node
 apply left rotation on imbalance node

Keys : 10, 30, 20



LR Imbalance : apply left rotation on left of imbalance node
 apply right rotation on imbalance node

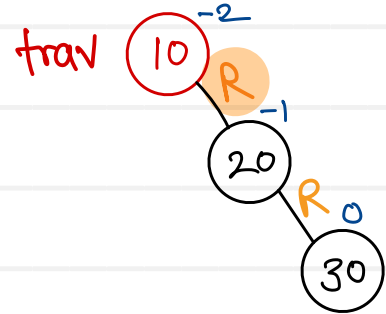
Keys : 30, 10, 20



$BF < -1$

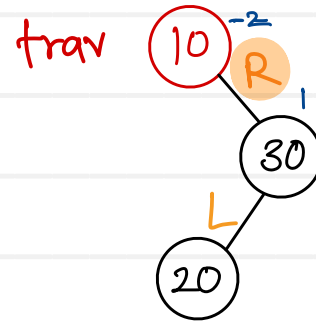
RR Imbalance

Keys : 10, 20, 30



RL Imbalance

Keys : 10, 30, 20



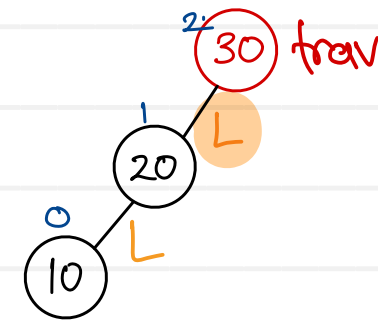
$value > trav.right.data$

$value < trav.right.data$

$BF > 1$

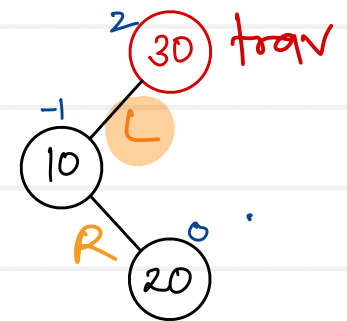
LL Imbalance

Keys : 30, 20, 10



LR Imbalance

Keys : 30, 10, 20

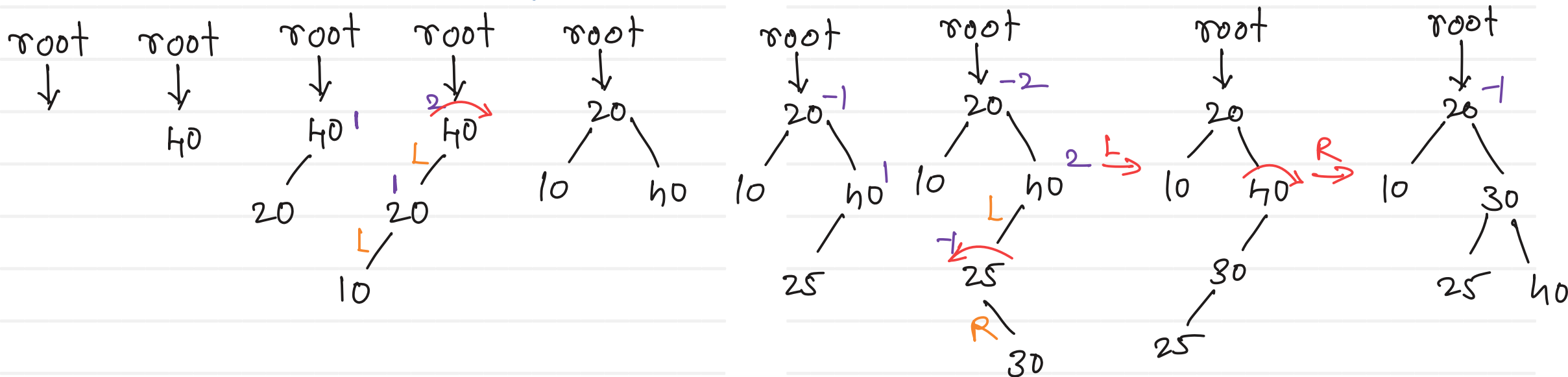


$value < trav.left.data$

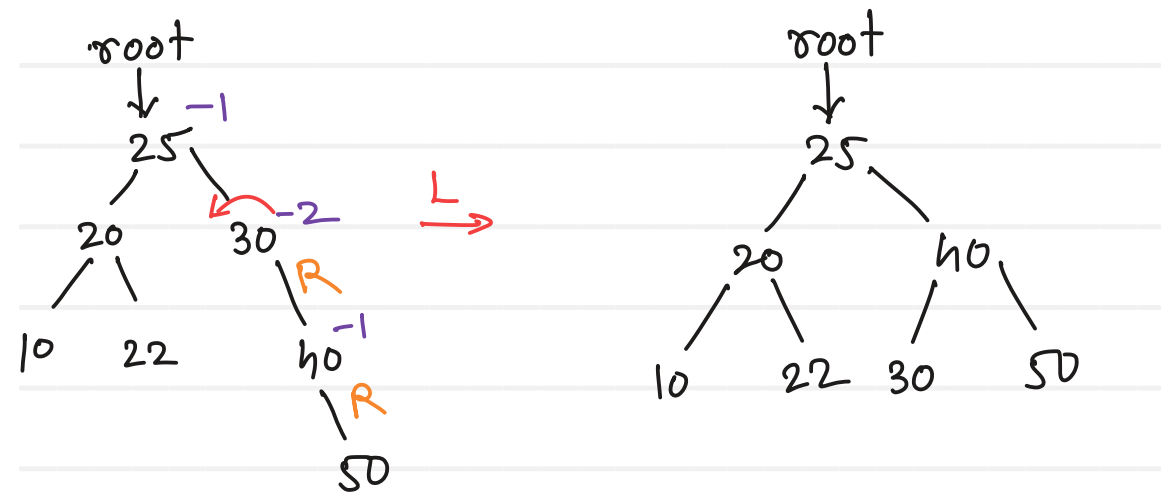
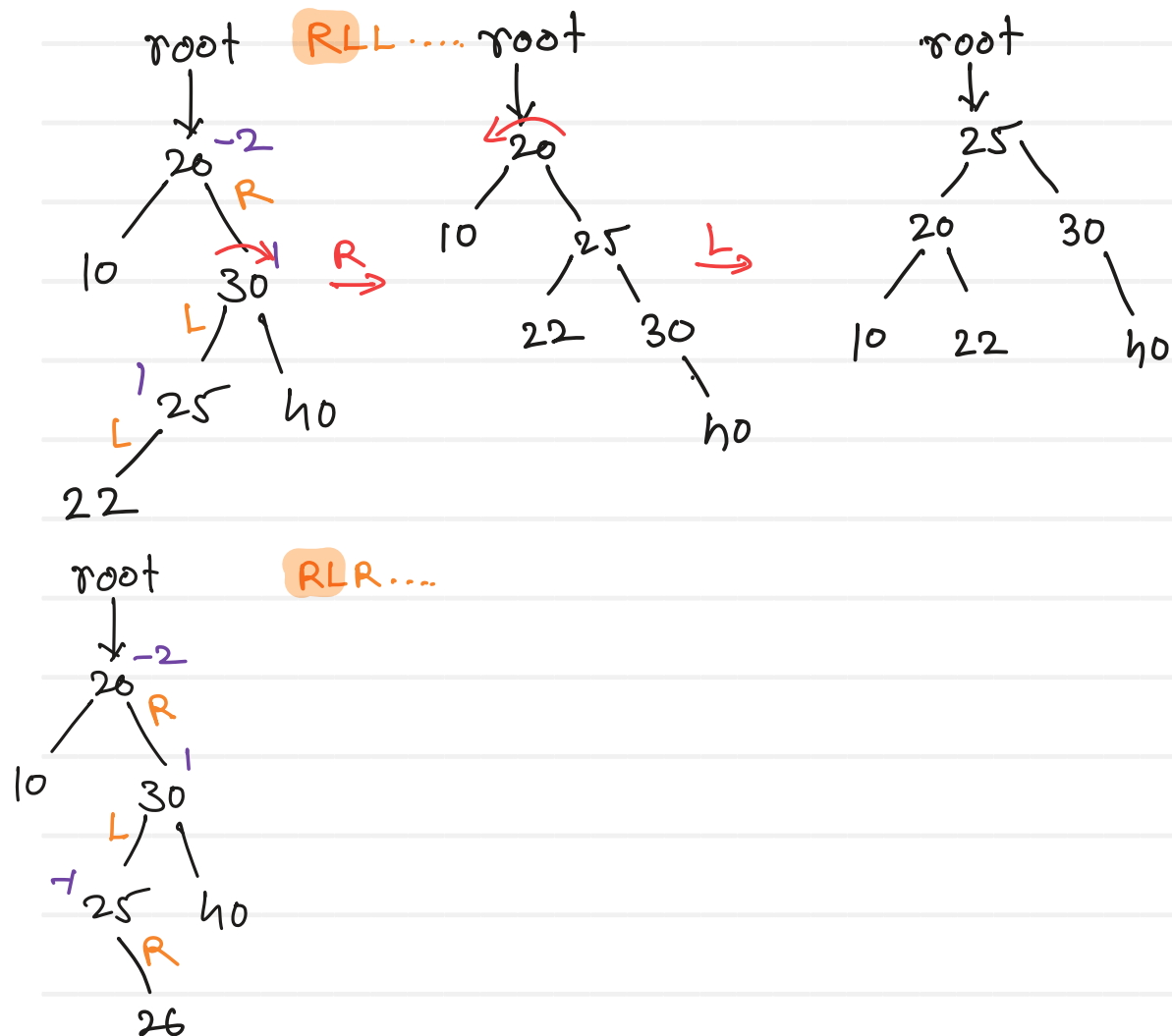
$value > trav.left.data$

- self balancing binary search tree
- on every insertion and deletion of a node, tree is getting balanced by applying rotations on imbalance nodes
- The difference between heights of left and right sub trees can not be more than one for all nodes
- Balance factors of all the nodes are either -1 , 0 or +1
- All operations of AVL tree are performed in $O(\log n)$ time complexity

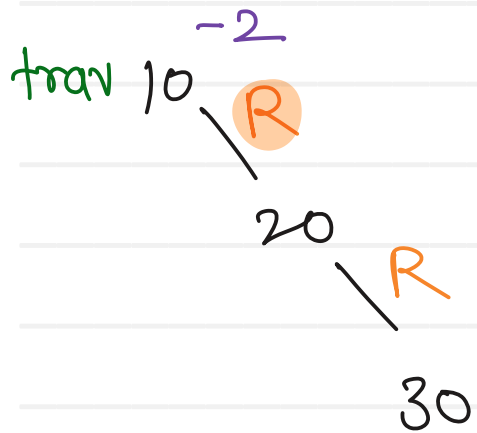
Keys : 40, 20, 10, 25, 30, 22, 50



Keys : 40, 20, 10, 25, 30, 22, 50

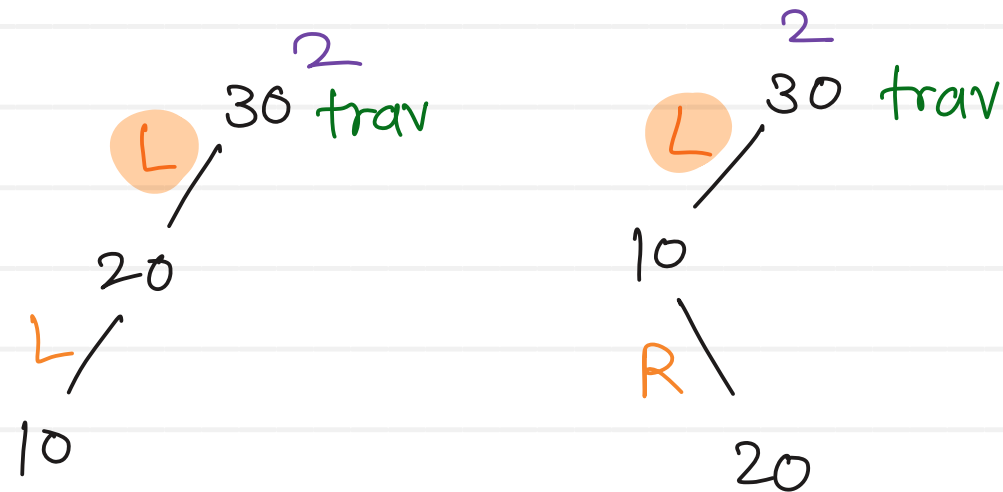


$BF < -1$



$value > trav.right.data$ $value < trav.right.data$

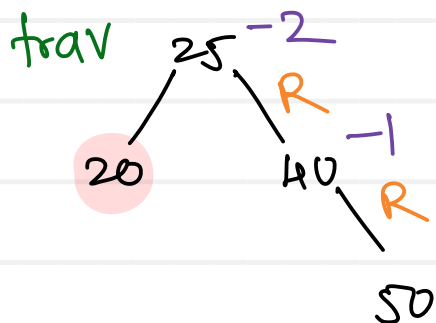
$BF > 1$



$value < trav.left.data$ $value > trav.left.data$

L-Deletion

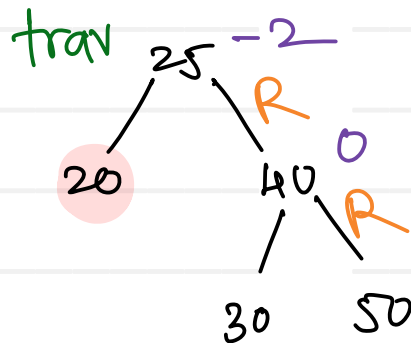
L-1



$$BF(trav) < -1$$

$$BF(trav.right) == -1$$

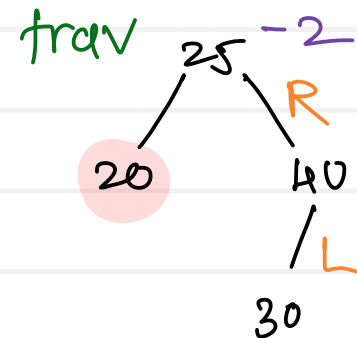
LO



$$BF(trav) < -1$$

$$BF(trav.right) == 0$$

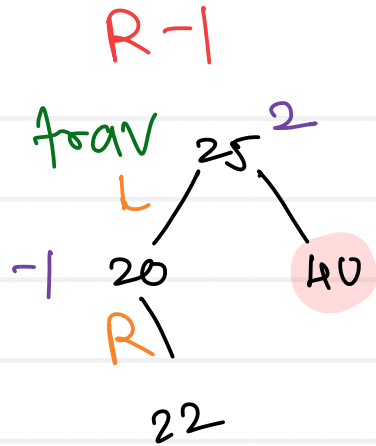
L+1



$$BF(trav) < -1$$

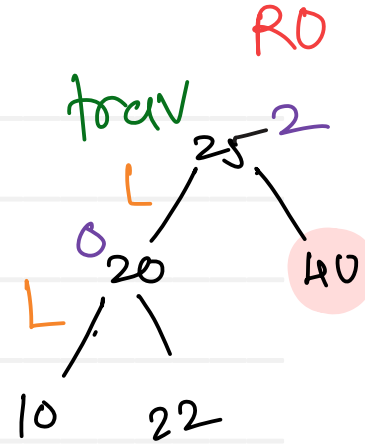
$$BF(trav.right) == +1$$

R-Deletion



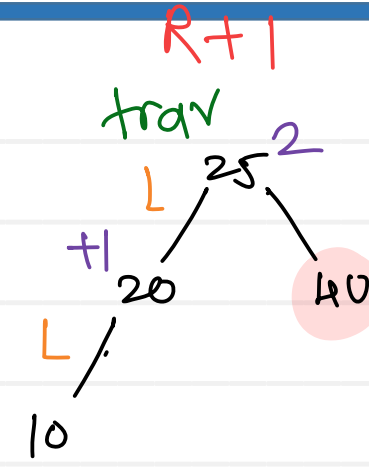
$$BF(trav) > 1$$

$$BF(trav.left) == -1$$



$$BF(trav) > 1$$

$$BF(trav.left) == 0$$



$$BF(trav) > 1$$

$$BF(trav.left) == +1$$

